

Model Training and Comparison for Multiclass Disease Classification

Model Training and Comparison for Multiclass Disease Classification Objective

This notebook focuses on training and comparing machine learning models for multiclass disease classification using the processed dataset prepared in the previous stage.

The main objectives are to:

establish a baseline benchmark, train a range of models from different model families, evaluate performance using metrics appropriate for imbalanced multiclass classification, compare models under a consistent data split and evaluation framework, identify the model that appears most suitable for subsequent analysis based on the observed results.

All preprocessing steps were completed in the previous notebook to maintain a clear separation between data preparation and model development.

Libraries and Modeling Tools

This notebook uses a set of standard Python libraries to support data handling, model training, and evaluation.

The libraries are organized by their primary function:

Data handling and visualization: pandas, NumPy, matplotlib, seaborn
Modeling and preprocessing: scikit-learn pipelines and scaling utilities
Machine learning models: linear, distance-based, tree-based, boosting, and neural network methods
Evaluation metrics: accuracy and F1-based metrics for multiclass performance assessment

This selection enables consistent training and comparison of models under a unified experimental framework.

```
In [7]: # Core libraries for data handling and visualization
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
import numpy as np

# Utilities for model pipelines and scaling
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Baseline and machine learning models
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.neighbors import KNeighborsClassifier
```

```

from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import StratifiedKFold
# Evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

# Plot style
plt.rcParams["figure.figsize"] = (12, 6)
sns.set_theme(style="whitegrid")

```

Loading the Processed Datasets

This notebook uses the processed training, validation, and test datasets generated in the previous stage.

The datasets were pre-cleaned, encoded, and split using stratified sampling to preserve class distribution.

Loading these datasets ensures consistency across experiments and allows all models to be evaluated under the same data conditions.

Utility Functions

To ensure consistency across notebooks, reusable functions are defined for loading processed datasets and separating features from the target variable.

This reduces repetition and helps maintain a consistent experimental setup across different modeling stages.

```

In [8]: def load_processed_data(path="../data/processed"):
        train = pd.read_csv(f"{path}/train.csv")
        val = pd.read_csv(f"{path}/val.csv")
        test = pd.read_csv(f"{path}/test.csv")
        return train, val, test

        def split_features_target(df, target_column="disease_encoded"):
            X = df.drop(columns=[target_column])
            y = df[target_column]
            return X, y

```

```

In [9]: # Load data
        train_df, val_df, test_df = load_processed_data()

        # Split features and target
        X_train, y_train = split_features_target(train_df)
        X_val, y_val = split_features_target(val_df)
        X_test, y_test = split_features_target(test_df)

```

In [23]: *# Basic sanity checks*

```
print("Train shape:", train_df.shape)
print("Validation shape:", val_df.shape)
print("Test shape:", test_df.shape)

print("\nFeature matrix shape (train):", X_train.shape)
print("Target vector shape (train):", y_train.shape)
```

Train shape: (69906, 175)

Validation shape: (14980, 175)

Test shape: (14980, 175)

Feature matrix shape (train): (69906, 174)

Target vector shape (train): (69906,)

Baseline Model: Dummy Classifier

A dummy classifier was used to establish a minimum performance benchmark.

This model predicts the most frequent class and does not learn from the feature set. It provides a useful reference for assessing whether trained models capture meaningful symptom–disease relationships.

Given the class imbalance, baseline evaluation focuses on:

Accuracy as a general reference, Weighted F1-score to reflect overall class-weighted performance, Macro F1-score to assess performance across all classes equally.

In this setting, Macro F1 is the most informative metric for judging whether a model performs beyond majority-class prediction.

```
In [10]: # Dummy baseline model (predicts most frequent class)
dummy_model = DummyClassifier(strategy="most_frequent", random_state=42)

# Train
dummy_model.fit(X_train, y_train)

# Predict on validation set
y_val_pred_dummy = dummy_model.predict(X_val)

# Evaluate
dummy_accuracy = accuracy_score(y_val, y_val_pred_dummy)
dummy_f1 = f1_score(y_val, y_val_pred_dummy, average="weighted")
dummy_f1_macro = f1_score(y_val, y_val_pred_dummy, average="macro")

print("Dummy Model Performance (Validation Set):")
print("Accuracy:", round(dummy_accuracy, 4))
print("Weighted F1 Score:", round(dummy_f1, 4))
print("Macro F1:", round(dummy_f1_macro, 4))
```

Dummy Model Performance (Validation Set):

Accuracy: 0.2133

Weighted F1 Score: 0.075

Macro F1: 0.0117

The dummy classifier achieved an accuracy of approximately 21%, which is consistent with the prevalence of the largest class.

However, both Weighted F1 (~0.075) and Macro F1 (~0.010) are very low, indicating that the model fails to provide useful performance beyond majority-class prediction.

This confirms two points:

the dataset is strongly imbalanced, accuracy alone is not sufficient for model assessment.

Any meaningful trained model should show clear improvement, particularly in Macro F1.

Reusable Evaluation Function

To ensure consistent comparison across models, a reusable evaluation function is used.

Each model is trained and evaluated under the same procedure, with performance reported using:

Accuracy Weighted F1-score Macro F1-score

Because the task is multiclass and imbalanced, Macro F1 is treated as the most informative comparison metric, while Accuracy and Weighted F1 provide supporting context.

```
In [11]: def evaluate_model(model, X_train, y_train, X_val, y_val, model_name):
# Train model
model.fit(X_train, y_train)

# Predict
y_val_pred = model.predict(X_val)

# Metrics
accuracy = accuracy_score(y_val, y_val_pred)
f1_weighted = f1_score(y_val, y_val_pred, average="weighted")
f1_macro = f1_score(y_val, y_val_pred, average="macro")

# Print summary (NEW)
print(f"\n{model_name}")
print(f"Accuracy: {accuracy:.4f}")
print(f"Weighted F1: {f1_weighted:.4f}")
print(f"Macro F1: {f1_macro:.4f}")

return {
    "Model": model_name,
    "Accuracy": accuracy,
    "Weighted F1": f1_weighted,
    "Macro F1": f1_macro
}, y_val_pred

results_list = []
```

Model Comparison Strategy

A range of models from different families was evaluated to assess how various learning approaches perform on the disease classification task.

The selected models include:

Linear models: Logistic Regression, Linear SVM Distance-based model: K-Nearest Neighbors (KNN) Tree-based model: Random Forest Boosting model: XGBoost Neural network model: MLPClassifier

This selection allows comparison across models with different assumptions about the data, including linear separability, local similarity, and non-linear feature interactions.

All models are trained and evaluated using the same data split and evaluation metrics to ensure a consistent and fair comparison.

The selected models represent different learning assumptions about the relationship between symptoms and diseases.

- Linear models (Logistic Regression, SVM):

Assume that diseases can be separated using linear combinations of symptoms. These models are efficient and provide a strong baseline for high-dimensional data.

- Distance-based model (KNN):

Assumes that patients with similar symptom patterns belong to similar disease classes. This evaluates whether local similarity is sufficient for classification.

- Tree-based model (Random Forest):

Captures non-linear relationships and interactions between symptoms, which is important when diseases are defined by combinations of features. Boosting model (XGBoost):

- Focuses on improving performance by sequentially correcting errors, often providing strong results in structured datasets.
- Neural network (MLP):

Learns complex, non-linear patterns without predefined assumptions about feature relationships.

Model Selection Process

Model configurations used in this notebook were informed by prior experimentation conducted in a separate exploratory notebook.

Different model families and parameter settings were evaluated to identify stable and reasonable configurations. Based on those results, a subset of representative models was selected for structured comparison in this stage.

The goal of this notebook is not exhaustive tuning, but consistent comparison across models using a fixed experimental setup.

Logistic Regression Model

Logistic Regression is used as the first trained model to establish a strong and interpretable baseline.

It assumes that diseases can be distinguished using linear combinations of symptom features, which is suitable given the binary nature of the input variables.

To address class imbalance, class weighting is applied (`class_weight="balanced"`), ensuring that minority disease classes contribute more during training.

The maximum number of iterations is increased to ensure convergence.

```
In [12]: # Logistic Regression model (with class balancing)
logistic_model = LogisticRegression(
    max_iter=2600,
    class_weight="balanced",
    random_state=42
)

# Evaluate
logistic_results, y_val_pred_logistic = evaluate_model(
    logistic_model,
    X_train, y_train,
    X_val, y_val,
    "Logistic Regression (Balanced)"
)

# Store results
results_list.append(logistic_results)

# Display
results_df = pd.DataFrame(results_list)
display(results_df)
```

```
Logistic Regression (Balanced)
Accuracy: 0.3335
Weighted F1: 0.3471
Macro F1: 0.2960
```

	Model	Accuracy	Weighted F1	Macro F1
0	Logistic Regression (Balanced)	0.333511	0.34706	0.295965

Logistic Regression shows a clear improvement over the baseline across all evaluation metrics.

Accuracy increases from approximately 21% to 33% Macro F1 improves substantially from near zero to approximately 0.3

This indicates that the model is able to learn meaningful relationships between symptoms and disease classes.

The relatively strong Macro F1 suggests improved performance across multiple classes, including minority classes, supported by the use of class weighting.

Overall, the results suggest that linear relationships capture a meaningful portion of the structure in the dataset.

Linear SVM Model

A Linear Support Vector Machine (SVM) is used as an alternative linear classification approach.

Unlike Logistic Regression, which models class probabilities, SVM focuses on finding decision boundaries that maximize separation between classes.

Class weighting is applied to address imbalance, and the regularization parameter is set to balance margin size and classification performance.

This model allows comparison between probability-based and margin-based linear methods.

```
In [13]: # Linear SVM model (your configuration)
svm_model = LinearSVC(
    C=1,
    class_weight="balanced",
    max_iter=3000,
    random_state=42
)

# Evaluate
svm_results, y_val_pred_svm = evaluate_model(
    svm_model,
    X_train, y_train,
    X_val, y_val,
    "Linear SVM (C=1, Balanced)"
)

# Store results
results_list.append(svm_results)

# Display updated comparison
results_df = pd.DataFrame(results_list)
display(results_df.sort_values(by="Macro F1", ascending=False))
```

```
Linear SVM (C=1, Balanced)
Accuracy: 0.3258
Weighted F1: 0.3400
Macro F1: 0.2891
```

	Model	Accuracy	Weighted F1	Macro F1
0	Logistic Regression (Balanced)	0.333511	0.347060	0.295965
1	Linear SVM (C=1, Balanced)	0.325768	0.340024	0.289121

The Linear SVM model achieves performance close to Logistic Regression across all evaluation metrics.

Accuracy decreases slightly (≈ 0.325 vs 0.333) Macro F1 is also marginally lower (≈ 0.289 vs 0.295)

This indicates that both models capture similar underlying structure in the data, suggesting that the relationship between symptoms and diseases is largely linear.

However, Logistic Regression consistently performs better, particularly in Macro F1. This suggests that modeling class probabilities provides a small advantage in handling overlapping symptom patterns across diseases.

Overall, linear models provide a strong and stable baseline for this task, with Logistic Regression showing slightly better balanced performance.

K-Nearest Neighbors (KNN)

K-Nearest Neighbors is used to evaluate a distance-based approach to classification.

This model assumes that patients with similar symptom patterns belong to the same disease class.

A pipeline with standard scaling is used to ensure that all features contribute equally to distance calculations.

```
In [14]: # Initialize model
knn_model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", KNeighborsClassifier(
        n_neighbors=5,
        weights="distance",
        n_jobs=-1
    ))
])

# Evaluate KNN model
knn_results, y_val_pred_knn = evaluate_model(
    knn_model,
    X_train, y_train,
    X_val, y_val,
    "KNN (k=5, distance-weighted)"
)

# Store results
results_list.append(knn_results)

# Create and display results table
results_df = pd.DataFrame(results_list)
results_df = results_df.sort_values(by="Macro F1", ascending=False)

display(results_df)
```

```
KNN (k=5, distance-weighted)
Accuracy: 0.2883
Weighted F1: 0.2827
Macro F1: 0.1929
```


	Model	Accuracy	Weighted F1	Macro F1
0	Logistic Regression (Balanced)	0.333511	0.347060	0.295965
1	Linear SVM (C=1, Balanced)	0.325768	0.340024	0.289121
2	KNN (k=5, distance-weighted)	0.288251	0.282694	0.192948

The KNN model performs noticeably worse than the linear models across all evaluation metrics.

Accuracy decreases to approximately 0.28 Macro F1 drops to approximately 0.28, indicating weaker performance across classes

This performance decline can be explained by two key factors:

- High dimensionality: With a large number of binary symptom features, distances between samples become less meaningful, reducing the effectiveness of nearest-neighbor selection.

- Symptom overlap across diseases: Many diseases share similar symptom patterns, meaning that patients with similar features may belong to different classes, weakening the core assumption of KNN.

These results suggest that local similarity alone is not sufficient for reliable disease classification in this dataset.

Random Forest Model

Random Forest is used to evaluate a tree-based ensemble method capable of capturing non-linear relationships between symptoms.

Unlike linear models, Random Forest can model interactions between multiple symptoms, which is relevant in clinical settings where diseases are defined by combinations of features.

Class imbalance is addressed using `class_weight="balanced_subsample"`, ensuring that each tree accounts for class distribution during training.

```
In [15]: # Initialize Random Forest model
rf_model = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    class_weight="balanced_subsample",
    n_jobs=-1
)

# Evaluate model
rf_results, y_val_pred_rf = evaluate_model(
    rf_model,
    X_train, y_train,
    X_val, y_val,
```

```

    "Random Forest (300 trees)"
)

# Store results
results_list.append(rf_results)

# Create and display results table
results_df = pd.DataFrame(results_list)
results_df = results_df.sort_values(by="Macro F1", ascending=False)

display(results_df)

```

Random Forest (300 trees)
 Accuracy: 0.4026
 Weighted F1: 0.3654
 Macro F1: 0.2378

	Model	Accuracy	Weighted F1	Macro F1
0	Logistic Regression (Balanced)	0.333511	0.347060	0.295965
1	Linear SVM (C=1, Balanced)	0.325768	0.340024	0.289121
3	Random Forest (300 trees)	0.402603	0.365405	0.237822
2	KNN (k=5, distance-weighted)	0.288251	0.282694	0.192948

Random Forest achieves higher accuracy compared to the linear models, reaching approximately 0.40.

However, its Macro F1 score (≈ 0.23) is slightly lower than that of Logistic Regression and SVM.

This indicates that:

The model improves overall classification performance, likely by capturing non-linear interactions between symptoms. However, it does not improve balanced performance across all classes.

This suggests that while non-linear relationships exist in the dataset, they do not consistently improve performance across minority classes.

Overall, Random Forest provides strong overall accuracy but does not outperform linear models in terms of balanced classification performance.

XGBoost Model

XGBoost is used to evaluate a boosting-based approach that builds models sequentially, focusing on correcting errors made by previous iterations.

This approach is well-suited for structured, tabular datasets and can capture complex, non-linear relationships between features.

```

In [16]: # Initialize XGBoost model
xgb_model = XGBClassifier(

```

```

n_estimators=300,
max_depth=8,
learning_rate=0.1,
subsample=0.8,
colsample_bytree=0.8,
random_state=42,
eval_metric="mlogloss"
)

# Evaluate model
xgb_results, y_val_pred_xgb = evaluate_model(
    xgb_model,
    X_train, y_train,
    X_val, y_val,
    "XGBoost"
)

# Store results
results_list.append(xgb_results)

# Create and display results table
results_df = pd.DataFrame(results_list)
results_df = results_df.sort_values(by="Macro F1", ascending=False)

display(results_df)

```

XGBoost
Accuracy: 0.6652
Weighted F1: 0.6550
Macro F1: 0.4592

	Model	Accuracy	Weighted F1	Macro F1
4	XGBoost	0.665154	0.655037	0.459176
0	Logistic Regression (Balanced)	0.333511	0.347060	0.295965
1	Linear SVM (C=1, Balanced)	0.325768	0.340024	0.289121
3	Random Forest (300 trees)	0.402603	0.365405	0.237822
2	KNN (k=5, distance-weighted)	0.288251	0.282694	0.192948

The XGBoost model achieves the strongest performance across all evaluation metrics.

Accuracy reaches approximately 0.665, significantly higher than all other models Macro F1 improves to approximately 0.459, indicating substantially better balanced performance across classes Weighted F1 (~0.655) is also the highest, confirming strong overall predictive performance

Compared to other models:

- vs Logistic Regression:

Macro F1 improves from ~0.296 to ~0.459, indicating a substantial gain in performance across minority classes

- vs Random Forest:

Both accuracy and Macro F1 are significantly higher, suggesting that boosting captures patterns more effectively than bagging

- vs Linear models:

The large performance gap indicates that the relationship between symptoms and diseases is not purely linear

These results suggest that:

Sequential error correction in XGBoost enables better learning of complex and overlapping symptom patterns. The model is more effective at distinguishing between classes with similar symptom profiles. Unlike previous models, XGBoost improves both overall accuracy and balanced performance simultaneously.

Overall, XGBoost appears to be the most effective model among those evaluated, providing the best balance between overall and per-class performance.

Neural Network Model

MLP Classifier

A Multi-Layer Perceptron (MLP) is used to evaluate a neural network approach for multiclass classification.

The model is capable of learning complex, non-linear relationships between symptoms and diseases without relying on predefined assumptions about feature interactions.

```
In [17]: # Initialize neural network pipeline
nn_model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", MLPClassifier(
        hidden_layer_sizes=(256, 128),
        max_iter=200,
        random_state=42,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=10
    ))
])

# Evaluate model
nn_results, y_val_pred_nn = evaluate_model(
    nn_model,
    X_train, y_train,
    X_val, y_val,
    "Neural Network"
)

# Store results
results_list.append(nn_results)

# Create and display results table
```

```
results_df = pd.DataFrame(results_list)
results_df = results_df.sort_values(by="Macro F1", ascending=False)

display(results_df)
```

Neural Network

Accuracy: 0.5513

Weighted F1: 0.5448

Macro F1: 0.3748

	Model	Accuracy	Weighted F1	Macro F1
4	XGBoost	0.665154	0.655037	0.459176
5	Neural Network	0.551268	0.544827	0.374786
0	Logistic Regression (Balanced)	0.333511	0.347060	0.295965
1	Linear SVM (C=1, Balanced)	0.325768	0.340024	0.289121
3	Random Forest (300 trees)	0.402603	0.365405	0.237822
2	KNN (k=5, distance-weighted)	0.288251	0.282694	0.192948

MLP Performance

The Neural Network (MLP) model shows strong performance, outperforming all models except XGBoost.

Accuracy reaches approximately 0.551, significantly higher than linear models and Random Forest Macro F1 improves to approximately 0.375, indicating substantially better performance across classes compared to earlier models Weighted F1 (~0.545) confirms strong overall predictive capability

Compared to other models:

- vs Logistic Regression / SVM:

The large improvement in both accuracy and Macro F1 indicates that non-linear relationships between symptoms and diseases are important

-vs Random Forest: The MLP achieves significantly higher Macro F1 (~0.375 vs ~0.238), suggesting better handling of class balance

- vs XGBoost:

XGBoost still performs better across all metrics (Macro F1 ~0.459), indicating that boosting is more effective at refining decision boundaries in this dataset

These results suggest that:

Neural networks are capable of capturing complex symptom interactions However, they do not outperform boosting methods, likely due to: structured/tabular nature of the data effectiveness of gradient boosting in handling such datasets

Overall, the MLP provides strong performance and demonstrates the importance of non-linear modeling, but XGBoost remains the most effective model in this comparison.

```

In [18]: # Create comparison table
results_df = pd.DataFrame(results_list).copy()

# Sort by primary metric
results_df = results_df.sort_values(
    by=["Macro F1", "Weighted F1", "Accuracy"],
    ascending=False
).reset_index(drop=True)

# Add rank
results_df.insert(0, "Rank", range(1, len(results_df) + 1))

# Round for readability
metric_cols = ["Accuracy", "Macro F1", "Weighted F1"]
results_df[metric_cols] = results_df[metric_cols].round(4)

# Add selection note
results_df["Selection Note"] = ""
if not results_df.empty:
    results_df.loc[0, "Selection Note"] = "Best overall balance"
    if len(results_df) > 1:
        results_df.loc[1, "Selection Note"] = "Strong alternative"

# Reorder columns
results_df = results_df[
    ["Rank", "Model", "Macro F1", "Weighted F1", "Accuracy", "Selection Note"]
]

# Save results
output_dir = Path("../results")
output_dir.mkdir(parents=True, exist_ok=True)
results_df.to_csv("../results/tables/model_comparison_results.csv", index=False)

# Display
results_df_display = results_df.set_index("Rank")
display(results_df_display)

```

	Model	Macro F1	Weighted F1	Accuracy	Selection Note
Rank					
1	XGBoost	0.4592	0.6550	0.6652	Best overall balance
2	Neural Network	0.3748	0.5448	0.5513	Strong alternative
3	Logistic Regression (Balanced)	0.2960	0.3471	0.3335	
4	Linear SVM (C=1, Balanced)	0.2891	0.3400	0.3258	
5	Random Forest (300 trees)	0.2378	0.3654	0.4026	
6	KNN (k=5, distance-weighted)	0.1929	0.2827	0.2883	

```

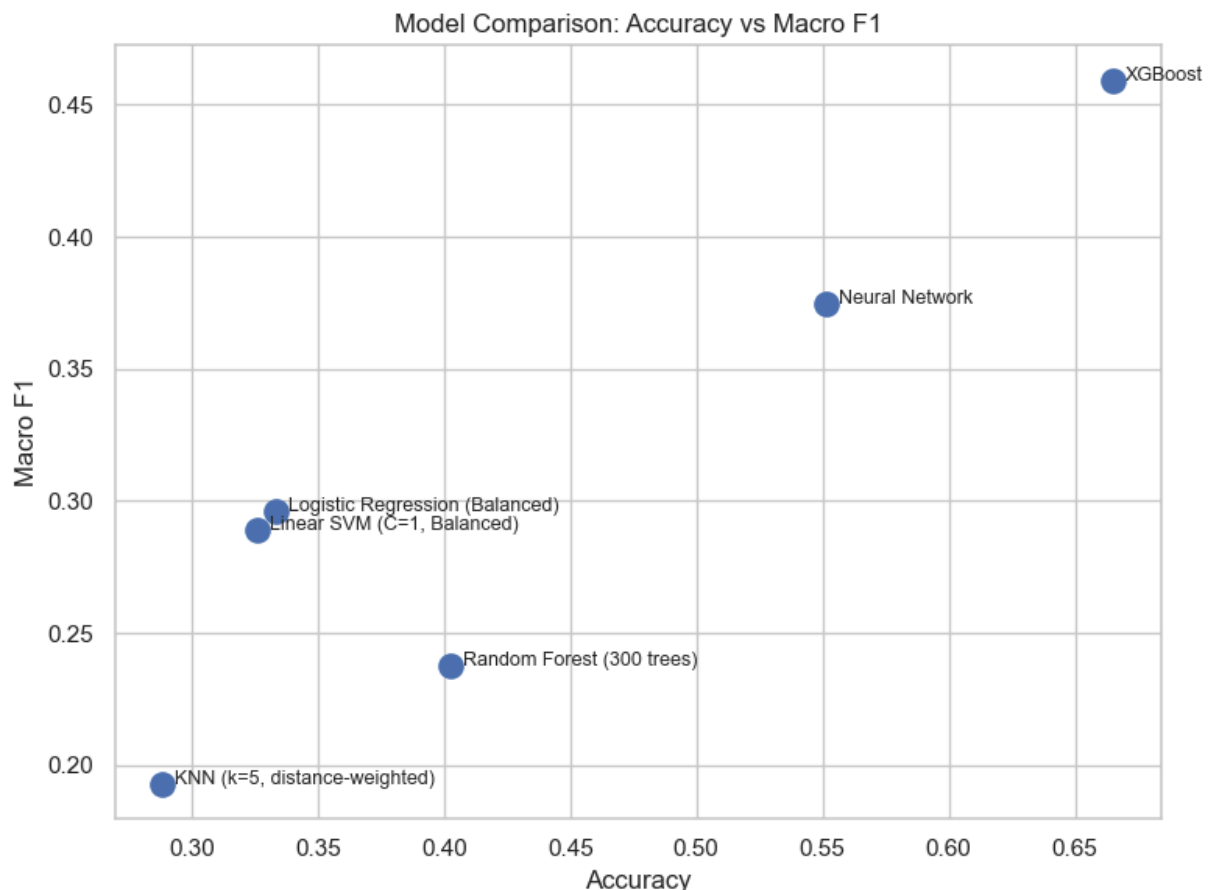
In [20]: plt.figure(figsize=(8, 6))
plt.scatter(results_df["Accuracy"], results_df["Macro F1"], s=120)

for _, row in results_df.iterrows():
    plt.text(row["Accuracy"] + 0.005, row["Macro F1"], row["Model"], fontsize=10)

```

```
plt.xlabel("Accuracy")
plt.ylabel("Macro F1")
plt.title("Model Comparison: Accuracy vs Macro F1")
plt.tight_layout()
plt.savefig("../results/graphs/model_comparison_accuracy_vs_macro_f1.png", c
plt.show()
# Best model
best_model_name = results_df.loc[0, "Model"]
best_macro_f1 = results_df.loc[0, "Macro F1"]

print(f"Best model based on Macro F1: {best_model_name}")
print(f"Best Macro F1 score: {best_macro_f1:.4f}")
```



Best model based on Macro F1: XGBoost
 Best Macro F1 score: 0.4592

Model Comparison Summary

The evaluated models show clear differences in performance across both overall accuracy and balanced classification.

- **XGBoost** achieves the highest performance across all metrics, with the strongest Macro F1, indicating the best balance across classes
- **Neural Network (MLP)** provides the second-best performance, demonstrating the importance of non-linear modeling
- **Logistic Regression and Linear SVM** establish strong linear baselines, but are limited in capturing complex relationships
- **Random Forest** improves overall accuracy but does not achieve strong balanced performance

- **KNN** performs the weakest, likely due to high dimensionality and overlapping symptom patterns

Given the evaluation strategy, where Macro F1 is the primary metric, XGBoost provides the most consistent performance across all classes.

The results suggest that:

- The relationship between symptoms and diseases is **non-linear and interaction-driven**
- Models that iteratively refine decision boundaries (e.g., boosting) are more effective in this setting
- Simpler models capture baseline structure but are insufficient for optimal performance

Why XGBoost Performed Better

XGBoost outperforms other models due to its underlying gradient boosting framework and several key properties:

- **Sequential Error Correction:** Unlike bagging methods such as Random Forest, XGBoost builds trees sequentially, where each new tree focuses on correcting the errors of the previous ones. This allows the model to better learn difficult patterns, especially in minority classes.
- **Ability to Model Complex Relationships:** XGBoost effectively captures non-linear relationships and feature interactions, which are likely present in this dataset. This explains its advantage over linear models such as Logistic Regression and SVM.
- **Regularization Mechanisms:** XGBoost includes built-in regularization (L1/L2), which helps prevent overfitting while maintaining high model capacity.
- **Robustness to Feature Scaling:** Unlike Neural Networks, XGBoost does not require feature scaling and is less sensitive to feature distributions, making it more stable in practice.
- **Handling of Imbalanced Data:** While not explicitly designed for imbalance, the boosting process naturally emphasizes harder-to-classify samples, which often correspond to minority classes. This contributes to its higher Macro F1 score.

Comparison with Other Models

- The **Neural Network** demonstrates moderate performance but is more sensitive to class imbalance and data size, limiting its ability to generalize across all classes.
- **Random Forest** performs worse than XGBoost, indicating that bagging alone is less effective than boosting for this problem.
- **Logistic Regression** and **SVM** are constrained by linear decision boundaries, which are insufficient for capturing the complexity of the data.
- **KNN** is negatively impacted by class imbalance and high dimensionality, resulting in the weakest performance.

Model Selection Summary

- XGBoost outperforms all models across Accuracy, Weighted F1, and Macro F1
- Linear models provide strong baseline performance but are limited by linear assumptions
- Tree-based models improve accuracy but do not sufficiently address class imbalance
- XGBoost achieves the best balance between overall performance and class-level consistency

Final Selection: XGBoost is selected as the final model for further analysis

Class-Level Evaluation

Following model comparison, XGBoost was selected as the final model based on its superior validation performance. However, aggregate metrics alone are insufficient to fully characterise performance in a multiclass classification setting.

To obtain a more granular evaluation, a classification report is generated on the validation set, providing precision, recall, and F1-score for each class. This enables a detailed assessment of the model's ability to correctly identify each disease category.

This analysis is particularly critical given the presence of class imbalance. It ensures that model performance is not dominated by majority classes and that predictive quality remains consistent across all classes.

```
In [21]: # Predictions from XGBoost
y_pred_xgb = y_val_pred_xgb

# Classification report
report = classification_report(
    y_val,
    y_pred_xgb,
    output_dict=True,
    zero_division=0
)

report_df = pd.DataFrame(report).transpose()

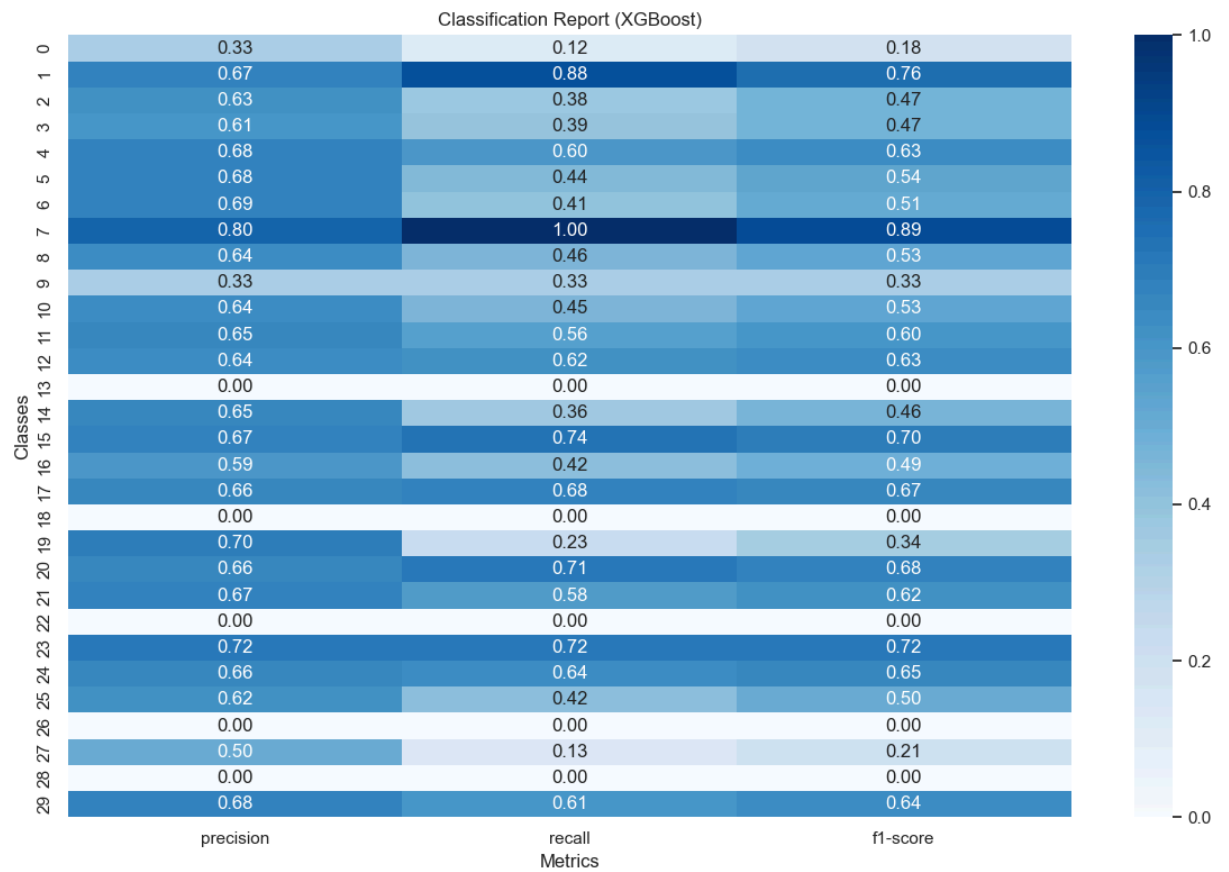
# ---- Clean heatmap ----
report_heatmap = report_df.iloc[:-3, :-1]

plt.figure(figsize=(12, 8))
sns.heatmap(report_heatmap, annot=True, fmt=".2f", cmap="Blues")

plt.title("Classification Report (XGBoost)")
plt.ylabel("Classes")
plt.xlabel("Metrics")

plt.tight_layout()
```

```
plt.savefig("../results/graphs/classification_report_xgboost.png")
plt.show()
```



- Performance varies significantly across classes
 1. Strong results are observed for high-support classes (e.g., 1, 15, 17, 20, 23, 24) with F1-scores above 0.65
 2. These classes are learned effectively due to sufficient training samples
- Minority classes are not learned
 1. Several low-support classes (e.g., 13, 18, 22, 26, 28) have zero precision and recall
 2. Indicates complete failure to detect these categories
- Recall is consistently lower than precision for many classes
 1. Observed in classes such as 2, 3, 5, 6, 14, 16
 2. Suggests the model misses a substantial number of true instances (under-detection)
- Class imbalance strongly impacts performance
 1. Model is biased toward majority classes
 2. Minority classes receive insufficient representation during training

To conclude:

1. The model demonstrates strong class-level performance for dominant classes
2. However, it lacks robustness across the full class distribution, particularly for underrepresented classes

Confusion Matrix Analysis

The confusion matrix provides a detailed view of prediction distribution across classes, highlighting both correct classifications and misclassification patterns.

Strong diagonal concentration is observed for majority classes, indicating accurate predictions. Minority classes show weak or sparse diagonal representation, confirming poor detection. Off-diagonal patterns suggest systematic misclassification between certain classes. Errors are more pronounced in underrepresented classes, reflecting the impact of class imbalance.

Overall, the confusion matrix reinforces the findings from the classification report, demonstrating strong performance on dominant classes and inconsistent performance on minority classes.

```
In [22]: # True labels and predictions
y_true = y_val
y_pred = y_val_pred_xgb

# Compute confusion matrix
cm = confusion_matrix(y_true, y_pred)

# Plot
```

```

fig, ax = plt.subplots(figsize=(10, 8))

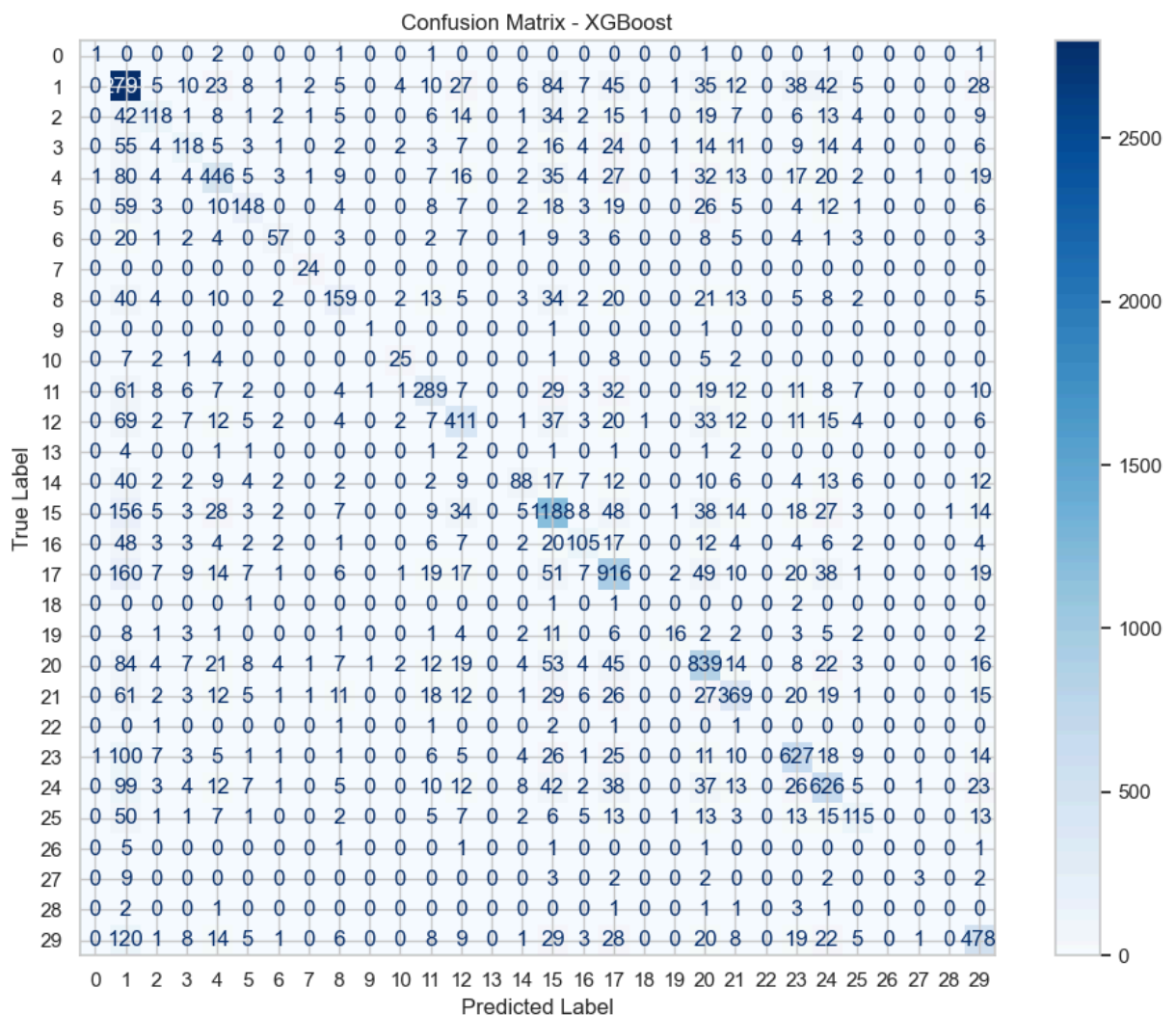
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm
)

disp.plot(ax=ax, cmap="Blues", colorbar=True)

plt.title("Confusion Matrix - XGBoost")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.tight_layout()
plt.savefig("../results/graphs/confusion_matrix_xgboost.png")
plt.show()

```



Confusion Matrix Analysis

- **Clustered misclassification among related classes**
 - Strong confusion observed between **Classes 2, 3, and 4**
 - These classes are frequently misclassified among each other, suggesting similar feature patterns
- **Mid-group overlap**
 - Notable confusion between **Classes 5 and 6**

- Indicates difficulty in distinguishing between these categories
 - **Sequential class confusion**
 - Classes **11 and 12** show consistent bidirectional misclassification
 - Suggests overlapping decision boundaries
 - **Large cluster confusion in dominant groups**
 - Significant overlap between **Classes 15, 16, and 17**
 - Errors within this group indicate high intra-group similarity
 - **Adjacent class confusion**
 - Classes **20 and 21** are frequently confused
 - Likely due to similar feature distributions
 - **Highly similar pair**
 - Strong bidirectional confusion between **Classes 23 and 24**
 - Indicates these classes are particularly difficult to separate
 - **Bias-driven misclassification**
 - Several minority classes (e.g., 13, 18, 22, 26, 28) are misclassified into dominant classes such as **1, 15, and 20**
 - Reflects model bias toward high-frequency labels
 - **Asymmetric misclassification**
 - Some classes are frequently predicted as another class, but not vice versa (e.g., smaller classes → larger classes)
 - Indicates uneven class boundaries and dominance effects
 - **Key insight**
 - Misclassifications are concentrated within specific class groups rather than being random, highlighting feature similarity and insufficient separability between certain classes
-

Conclusion

This notebook presented the training and evaluation of multiple machine learning models for multiclass disease classification. A range of models, including linear, distance-based, tree-based, ensemble, and neural network approaches, were compared using consistent training and validation splits.

Among the evaluated models, XGBoost achieved the strongest performance, particularly in terms of Macro F1-score, which was selected as the primary metric due to class imbalance. This indicates that XGBoost provides the most balanced performance across the different disease classes.

A detailed evaluation using classification reports, per-class F1-score analysis, and confusion matrices revealed that while the model performs well on majority classes, it struggles to correctly classify minority classes with limited representation. This

behaviour reflects the significant class imbalance present in the dataset. However, improving performance on underrepresented classes remains a key challenge and motivates further refinement in the next stage of the study.

Limitations

- Model performance remains sensitive to class imbalance
- Several minority classes are still underrepresented
- Current evaluation is based on fixed hyperparameters without exhaustive tuning